

Documentacao Tecnica - Escola High Tech Frontend

Versao: 0.2.0 Data de revisao: 2026-05-28

1. Objetivo

Este documento descreve a arquitetura, os modulos, os contratos e os pontos de manutencao do frontend da aplicacao Escola High Tech.

O frontend e uma SPA Nuxt 3 que consome uma API REST externa. Ele controla a experiencia de usuario, roteamento, autenticao no navegador, validacoes locais, permissoes visuais e integracoes de tela. A API continua sendo a fonte final de dados, autorizacao e persistencia.

As funcionalidades de QR Code bancario para alunos e geracao de PDF de holerite no cliente ficam no front. Holerites salvos, listados, baixados e excluidos usam os endpoints da API.

2. Stack tecnica

Tecnologia	Responsabilidade
Nuxt 3.21.6	Framework Vue, roteamento por arquivos, plugins e build
Vue 3	Componentes e Composition API
TypeScript strict	Tipagem de contratos, paginas e utilitarios
Pinia	Estado global, principalmente autenticao
Tailwind CSS	Estilizacao utilitaria
ofetch / \$fetch	Cliente HTTP
@lucide/vue	Iconografia
qrcode	Geracao de QR Code no cliente
jsPDF	Geracao de PDF de holerite no cliente
Vitest + happy-dom	Testes unitarios
Nginx	Servir build estatico no Docker

3. Configuracao Nuxt

Arquivo principal: `nuxt.config.ts` .

Configuracoes relevantes:

- `ssr: false` : aplicacao gerada como SPA.
- `modules` : `@pinia/nuxt` e `@nuxtjs/tailwindcss` .
- `css` : `~/assets/css/main.css` .
- `runtimeConfig.public.apiBase` : controla a URL base da API.
- `app.baseUrl` : permite deploy em subcaminho.
- `typescript.strict: true` .

Variaveis:

Variavel	Uso
NEXT_PUBLIC_API_BASE	URL publica/base da API
NEXT_APP_BASE_URL	Base URL da aplicacao

4. Estrutura de diretorios

```
assets/css/main.css      # Tailwind e estilos globais
components/              # Componentes reutilizaveis
docs/                    # Documentacao tecnica
layouts/                  # Layouts default e auth
middleware/auth.global.ts # Middleware global de autenticacao
middleware/aluno.ts       # Restricao para rotas exclusivas de alunos
middleware/funcionario.ts # Restricao para rotas exclusivas de funcionarios
pages/                    # Rotas Nuxt
plugins/api.ts            # Plugin que injeta $api
stores/auth.ts            # Store de autenticacao
tests/                    # Testes unitarios
types/api.ts              # Contratos TypeScript da API
utils/                    # Utilitarios compartilhados
```

5. Autenticacao

Arquivos:

- stores/auth.ts
- middleware/auth.global.ts
- plugins/auth.client.ts
- plugins/api.ts
- utils/api-client.ts

Fluxo:

1. O usuario autentica em `/login` .
2. A resposta de login e persistida no `localStorage` com a chave `form-escola-auth` .
3. O middleware global carrega a sessao, valida via `/auth/me` e protege rotas privadas.
4. O plugin `$api` injeta o token JWT no header `Authorization` .
5. Em resposta `401` , a sessao e encerrada e o usuario volta para `/login` .

Regras do middleware:

- Rotas publicas nao exigem token.
- Usuario autenticado que acessa `/login` volta ao painel.
- Usuario com senha padrao e direcionado a `/alterar-senha` .
- Rotas com `meta.roles` verificam `usuario.descricaoPerfil` .

6. Perfis e permissoes

Utilitario principal: `utils/usuario-permissions.ts` .

Perfis normalizados:

- administrador
- diretoria
- professor
- aluno
- desconhecido

O front usa essas permissões para esconder botoões, campos e ações. A API deve repetir as regras para segurança real.

7. Modulos de tela

7.1 Login e senha

Rotas:

- /login
- /alterar-senha

Recursos:

- Login via API.
- Reset/esqueci senha.
- Troca obrigatória quando deveAlterarSenhaPadrao estiver ativo.
- Medidor de força de senha.

7.2 Painel inicial

Rota: / .

Exibe atalhos para:

- Usuarios.
- Caderneta Digital.
- Calendario Escolar.
- QR Code, somente para aluno.
- Holerite, somente para professor e administrador.
- Alterar senha.

7.3 Usuarios

Rotas:

- /usuarios
- /usuarios/novo
- /usuarios/:id

Funcionalidades:

- Cadastro e edição de usuário.
- Busca e filtro por perfil.
- Controle de permissões por perfil.
- Upload e visualização de foto.
- Upload, download e exclusão de certificados PDF.
- Popup de contatos.

- Popup de documentos.
- Envio manual de notificacao.
- Campo `dataNascimento` com DatePicker.

Campo de aniversario:

- Componente: `components/DatePicker.vue` .
- Entrada digitavel: `dd/mm/aaaa` .
- Valor salvo no estado/payload: `yyyy-mm-dd` .
- Valor vazio enviado como `null` .
- Requer suporte do backend para persistencia definitiva.

7.4 Caderneta Digital

Rota: `/caderneta-digital` .

Funcionalidades:

- Cadastro, edicao e exclusao de disciplinas.
- Lancamento de notas, presencas e faltas.
- Consulta por disciplina.
- Professores administram lancamentos.
- Alunos visualizam apenas registros associados ao proprio cadastro.

7.5 Notificacoes

Implementadas no layout principal.

Funcionalidades:

- Listagem de notificacoes.
- Contador de nao lidas.
- Modal de detalhe.
- Marcar notificacao como lida.
- Marcar todas como lidas.
- Quebra de mensagens longas, URLs de fotos e URLs de PDFs sem overflow horizontal.

7.6 QR Code bancario ficticio

Rota: `/qr-code-bancario` .

Arquivos:

- `pages/qr-code-bancario.vue`
- `middleware/aluno.ts`
- `utils/qr-code-bancario.ts`

Funcionalidades:

- Disponivel somente para alunos.
- Gera dados bancarios demonstrativos por aluno.
- Gera QR Code localmente.
- Compartilha texto por WhatsApp.
- Abre e-mail via `mailto` .
- Copia dados.
- Baixa PNG do QR Code.

Todo payload gerado contém `SEM VALOR BANCARIO`.

7.7 Calendario Escolar

Rota: `/calendario-escolar`.

Arquivos:

- `pages/calendario-escolar.vue`
- `utils/feriados-brasil.ts`
- `utils/date-utils.ts`

Funcionalidades:

- Exibe calendario anual.
- Seleciona o mes vigente por padrao.
- Lista feriados nacionais brasileiros.
- Exibe grade mensal detalhada.
- Permite professor marcar avaliacoes e trabalhos por disciplina.
- Permite editar/excluir eventos.
- Mantem alunos e demais perfis sem permissao de gerenciamento em modo somente leitura.

Persistencia atual:

- A agenda de avaliacoes/trabalhos usa `localStorage`.
- A chave e separada por usuario autenticado.
- Para uso multiusuario real, criar endpoints no backend.
- Para notificar alunos matriculados quando o professor marcar avaliacao ou trabalho, o backend deve relacionar disciplina, matriculas e notificacoes.

Feriados:

- Feriados fixos nacionais.
- Paixao de Cristo calculada a partir da Pascoa.
- Dia Nacional de Zumbi e da Consciencia Negra incluido como feriado nacional.

7.8 Holerite

Rota: `/holerite`.

Arquivos:

- `pages/holerite.vue`
- `middleware/funcionario.ts`
- `utils/holerite-ficticio.ts`

Funcionalidades:

- Disponivel somente para professor e administrador.
- Professor consulta somente os proprios holerites retornados por `GET /holerites/me`.
- Administrador seleciona professor/administrador, lança rubricas editaveis e gera PDF no cliente.
- Administrador salva o PDF gerado em `POST /holerites/usuarios/{usuarioId}` usando `multipart/form-data`.
- Administrador lista, baixa e exclui holerites de funcionarios permitidos.
- Professor e administrador baixam PDF, exportam e compartilham dados por e-mail/WhatsApp.

Rubricas padrao sugeridas:

- Salario base.
- Gratificacao pedagogica ou administrativa.
- Auxilio alimentacao.
- Auxilio transporte.
- INSS demonstrativo.
- IRRF demonstrativo.
- Vale transporte.
- Plano de saude.
- Contribuicao associativa.
- FGTS apenas informativo.

Observacoes:

- Os valores sao preenchidos no front pelo administrador antes da geracao do PDF.
- A API atual recebe e persiste o PDF; nao recebe rubricas estruturadas em JSON.
- E-mail/WhatsApp abrem clientes externos com mensagem e link publico quando o storage retornar `url`.
- Envio server-side real com anexo/auditoria precisa de endpoint dedicado no backend.

8. Componentes reutilizaveis

DatePicker

Arquivo: `components/DatePicker.vue`.

Responsabilidades:

- Entrada manual com mascara `dd/mm/aaaa`.
- Conversao para `yyyy-mm-dd`.
- Selecao visual em calendario mensal.
- Navegacao de mes.
- Acoes Hoje, Limpar e OK.
- Estado `disabled` e `required`.

9. Utilitarios

Arquivo	Responsabilidade
<code>utils/api-client.ts</code>	Cliente HTTP e normalizacao de erros
<code>utils/api-file.ts</code>	Download de blobs protegidos por token
<code>utils/api-url.ts</code>	Resolucao de URLs da API
<code>utils/br-phone.ts</code>	Mascara e normalizacao de telefone
<code>utils/caderneta-digital.ts</code>	Parse de notas e regras locais da caderneta
<code>utils/date-utils.ts</code>	Mascara, parse e formatacao de datas
<code>utils/feriados-brasil.ts</code>	Feriados nacionais brasileiros
<code>utils/holerite-ficticio.ts</code>	Rubricas, totais, resumo e geracao de PDF de holerite
<code>utils/password-strength.ts</code>	Regras de forca de senha
<code>utils/qr-code-bancario.ts</code>	Dados ficticios e payload de QR Code

utils/usuario-permissions.ts	Regras visuais de permissao
utils/usuario-validation.ts	Validacao de e-mail duplicado

10. Contratos de API

Arquivo: `types/api.ts` .

Principais interfaces:

- `UsuarioSummary`
- `UsuarioForm`
- `UsuarioCreate`
- `UsuarioUpdate`
- `UsuarioArquivo`
- `Notificacao`
- `Perfil`
- `AuthResponse`
- `DisciplinaCaderneta`
- `CadernetaDigitalSummary`
- `Holerite`

Campo novo:

- `dataNascimento?: string | null` em `usuarios`.

Funcionalidades locais sem contrato de API:

- QR Code bancario ficticio.
- Rubricas editaveis usadas para gerar PDF de holerite antes do upload.
- Agenda escolar de avaliacoes/trabalhos enquanto persistir em `localStorage` .

11. Endpoints consumidos

Modulo	Endpoints
Auth	POST <code>/auth/login</code> , GET <code>/auth/me</code> , POST <code>/auth/alterar-senha</code> , POST <code>/auth/esqueci-senha</code>
Usuarios	GET <code>/usuarios</code> , GET <code>/usuarios/:id</code> , POST <code>/usuarios</code> , PUT <code>/usuarios/:id</code> , DELETE <code>/usuarios/:id</code> , GET <code>/usuarios/perfis</code>
Arquivos	GET <code>/usuarios/:id/arquivos</code> , GET <code>/usuarios/:id/foto</code> , POST <code>/usuarios/:id/foto</code> , POST <code>/usuarios/:id/certificados</code> , GET <code>/usuarios/:id/arquivos/:arquivoId/download</code> , DELETE <code>/usuarios/:id/arquivos/:arquivoId</code>
Notificacoes	GET <code>/notificacoes</code> , POST <code>/notificacoes</code> , PATCH <code>/notificacoes/:id/lida</code> , PATCH <code>/notificacoes/lidas</code>
Caderneta	GET <code>/caderneta-digital</code> , POST <code>/caderneta-digital</code> , PUT <code>/caderneta-digital/:id</code> , DELETE <code>/caderneta-digital/:id</code>
Disciplinas	GET <code>/caderneta-digital/disciplinas</code> , POST <code>/caderneta-digital/disciplinas</code> , PUT <code>/caderneta-digital/disciplinas/:id</code> , DELETE <code>/caderneta-digital/disciplinas/:id</code>
Holerites	GET <code>/holerites/me</code> , GET <code>/holerites/me/:id/download</code> , GET <code>/holerites/usuarios/:usuarioId</code> , POST <code>/holerites/usuarios/:usuarioId</code> , GET <code>/holerites/usuarios/:usuarioId/:id/download</code> , DELETE

```
/holerites/usuarios/:usuarioId/:id
```

12. Validacoes

Validacoes locais:

- HTML nativo (`required` , `type` , `maxlength`).
- `utils/br-phone.ts` para telefone.
- `utils/date-utils.ts` para data.
- `utils/password-strength.ts` para senha.
- `utils/usuario-validation.ts` para e-mail duplicado.
- `normalizeApiError` para mensagens vindas da API.

13. Testes

Comando:

```
npm run test
```

Cobertura:

- Store de autenticacao.
- Cliente API.
- Telefone brasileiro.
- Caderneta digital.
- Date utils.
- Feriados brasileiros.
- Holerite, totais e geracao de PDF.
- Forca de senha.
- QR Code bancario ficticio.
- Permissoes de usuario.
- Validacao de usuario.

14. Build e deploy

Comandos:

```
npm run typecheck  
npm run test  
npm run build  
npm run generate
```

Deploy estatico:

- Usar `.output/public` .
- Configurar rewrite de SPA para `index.html` .
- Definir `NUXT_PUBLIC_API_BASE` .

Docker:

- Build com Node 22.
- Geracao estatica.

- Servido por `nginx:alpine` .

15. Pendencias e integrações futuras

1. Persistir `dataNascimento` no backend.
2. Criar endpoints de agenda escolar:
 - `GET /agenda-escolar`
 - `POST /agenda-escolar`
 - `PUT /agenda-escolar/:id`
 - `DELETE /agenda-escolar/:id`
3. Sincronizar agenda entre professores/alunos.
4. Disparar notificações para alunos matriculados quando o professor marcar avaliação ou trabalho.
5. Criar endpoints de envio real de holerite por e-mail/WhatsApp caso seja necessário envio server-side com anexo/auditoria.
6. Opcionalmente criar envio real de e-mail/WhatsApp para QR Code.
7. Opcionalmente mover feriados para endpoint configurável, caso haja feriados estaduais/municipais.

16. Manutenção

Ao adicionar novas telas:

- Atualizar `pages/index.vue` se precisar aparecer no painel.
- Atualizar `layouts/default.vue` se precisar título dinâmico.
- Atualizar `types/api.ts` se houver contrato de API.
- Criar utilitário testável quando houver regra compartilhada.
- Atualizar README e esta documentação técnica.